University of Eloued

Faculty of Exact Sciences — Department of Computer Science

Module: Big Data Analytics | **Master 2:** IA & Data Science

Academic Year: 2026/2027

TP05: Real-Time Data Streaming and Processing Using Apache Kafka

Lab Report & Answer Key

1 Overview & Lab Setup

This laboratory assignment demonstrates the deployment and execution of a real-time event streaming pipeline using Apache Kafka , Docker Compose, and Python ('kafka-python').

1.1 System Architecture Summary

- **Container Environment:** Single-node Kafka broker (`apache/kafka:4.4.0`) executing in KRaft mode exposing port `9092`.
- **Topic Configuration:** `campus-sensors` topic with 3 partitions and a replication factor of 1.
- **Producer Service (`producer.py`):** Generates JSON-formatted sensor telemetry events containing timestamp, sensor identifier, temperature, and humidity.
- **Consumer Service (`consumer.py`):** Subscribes to `campus-sensors` under consumer group `tp05-campus-monitor`, deserializes JSON messages, prints event flows, and alerts on temperatures $\geq 35°C$.

2 Answers to Lab Questions

Q1: What problem does Apache Kafka solve in a data-streaming architecture?

Apache Kafka decouples data producers (event sources) from data consumers (event sinks). In traditional point-to-point architectures, applications directly communicate via API calls or shared static databases, creating tight coupling, performance bottlenecks, and single-point-of-failure vulnerabilities when message loads scale. Kafka provides an asynchronous, persistent, fault-tolerant distributed pub/sub commit log capable of handling millions of real-time events per second with high availability and arbitrary retention periods.

Q2: Key Kafka Definitions

- **Producer:** Client application that writes/publishes streams of events to Kafka topics.
- **Consumer:** Client application that subscribes to topics and reads/processes event logs.

- **Broker:** A single Kafka server node responsible for receiving, storing, replicating, and serving messages.
- **Topic:** A named stream or category to which messages are published and organized.
- **Partition:** An ordered, immutable segment sequence within a topic providing parallelism and scalability.
- **Offset:** A unique sequence integer assigned to each message within a partition, marking its position.
- **Consumer Group:** A logical collection of consumers working together to read and divide partitions from one or more topics.

Q3: What is the difference between a Kafka topic and a partition?

A **topic** is a logical entity representing an entire category or stream of data. A **partition** is a physical unit of data storage and parallelization underneath a topic. A single topic is split across one or more partitions. Messages within a partition are strictly ordered by sequential offsets, whereas global ordering across distinct partitions is not guaranteed unless explicit partition keying is configured.

Q4: Why does the consumer use `auto_offset_reset='earliest'`? When does this setting apply?

The setting `auto_offset_reset='earliest'` ensures that when a consumer group connects to a topic without an established or valid committed offset (for example, during initial startup or offset expiry), it reads messages starting from the very beginning of the partition log rather than ignoring past data. It applies only when no offset has been saved for that consumer group or when the requested offset has been deleted by log retention policies.

Q5: What is the purpose of the consumer group ID? What happens if a new group ID is used?

The consumer group ID identifies a set of consumer instances sharing the reading work for a topic. Kafka automatically balances the topic's partitions across all active consumer instances within the same group. If a brand-new group ID is introduced, Kafka treats it as an independent reader; it starts consuming events according to its `auto_offset_reset` policy (e.g., re-reading all historical events from the topic offset beginning without disturbing existing groups).

Q6: Why is JSON a suitable format for this exercise? Give one limitation.

JSON is lightweight, human-readable, semi-structured, natively supported by Python and web services, and schema-flexible for evolving sensor attributes. **Limitation:** JSON text payloads incur higher serialization/deserialization CPU overhead and larger memory/network footprints compared to binary formats like Apache Avro or Protocol Buffers, and lack strict baseline binary schema validation.

Q7: Explain how Docker Compose simplifies deployment compared with manual setup.

Docker Compose automates multi-container runtime specifications (environment variables, port mappings, volumes, network links, and service health dependencies) inside a single declarative

`compose.yaml` file. Instead of executing manual step-by-step terminal installation, binary configurations, environment variable definitions, and daemon startup routines, standard deployment requires only executing `docker compose up -d`.

Q8: What are the limitations of this single-broker educational setup compared with a production Kafka cluster?

1. **Zero Fault Tolerance:** High availability requires multiple broker nodes across fault domains. If the single broker container stops, the cluster goes down completely.
2. **No Data Redundancy:** Topics are constrained to a replication factor of 1, meaning physical hardware failure results in unrecoverable data loss.
3. **Throughput Bottlenecks:** Partition processing and concurrent I/O throughput are restricted to the resource limits of a single host.

3 Experimental Evidence & Screenshots

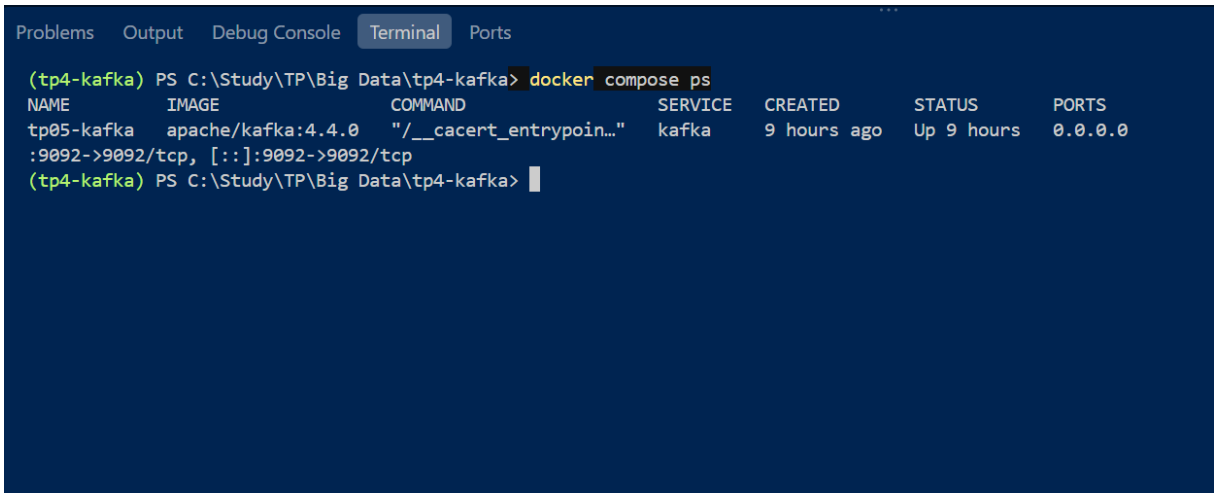


Figure 1: Verification of running Kafka broker container in KRaft mode.

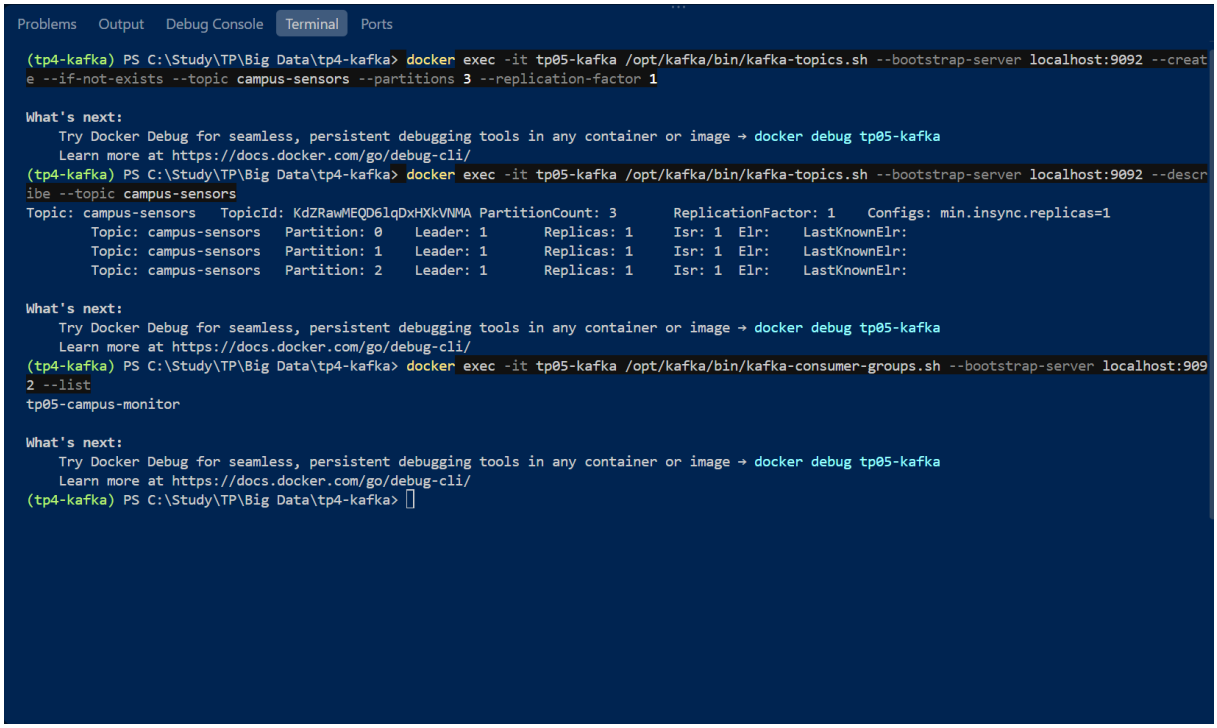


Figure 2: Kafka topic details showing 3 partitions and replication factor 1.

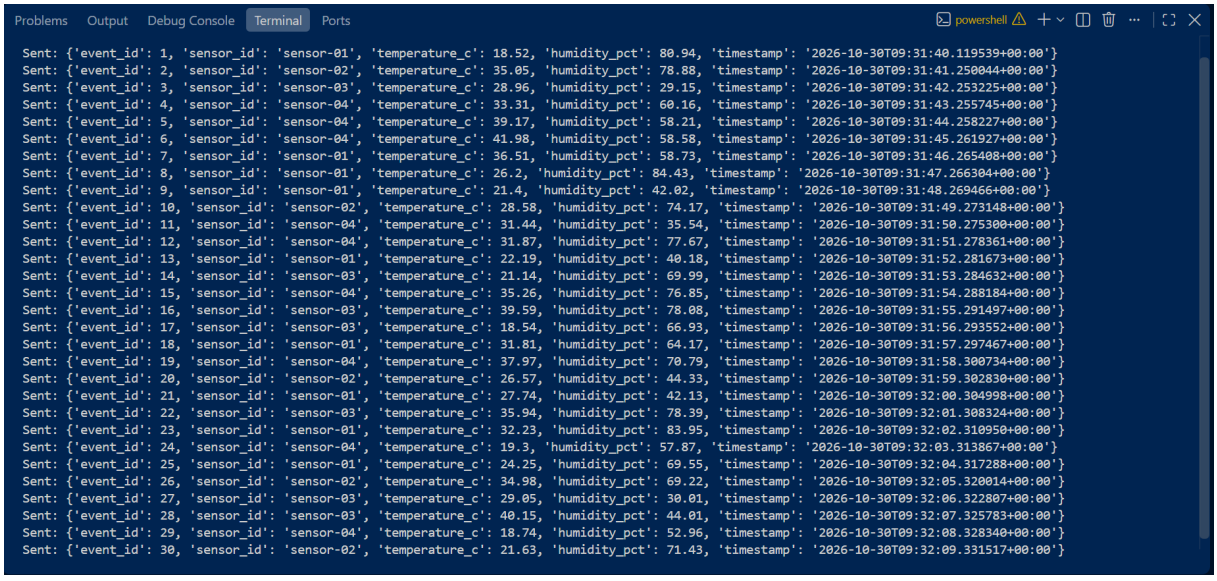


Figure 3: Terminal log of Python producer publishing simulated sensor readings.

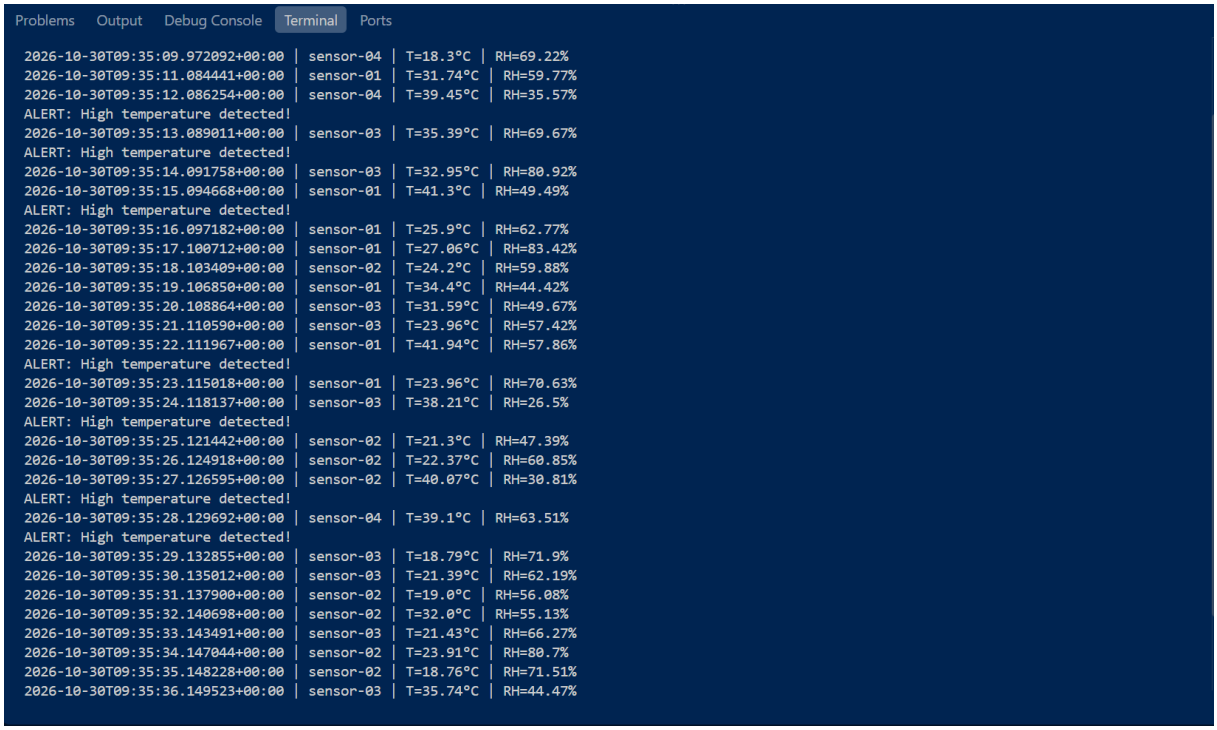


Figure 4: Terminal log of Python consumer receiving messages and alerting on $T \geq 35°$C.